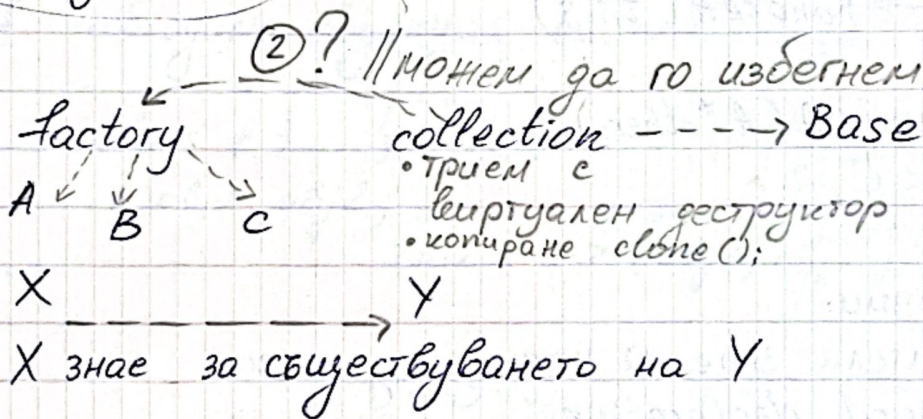
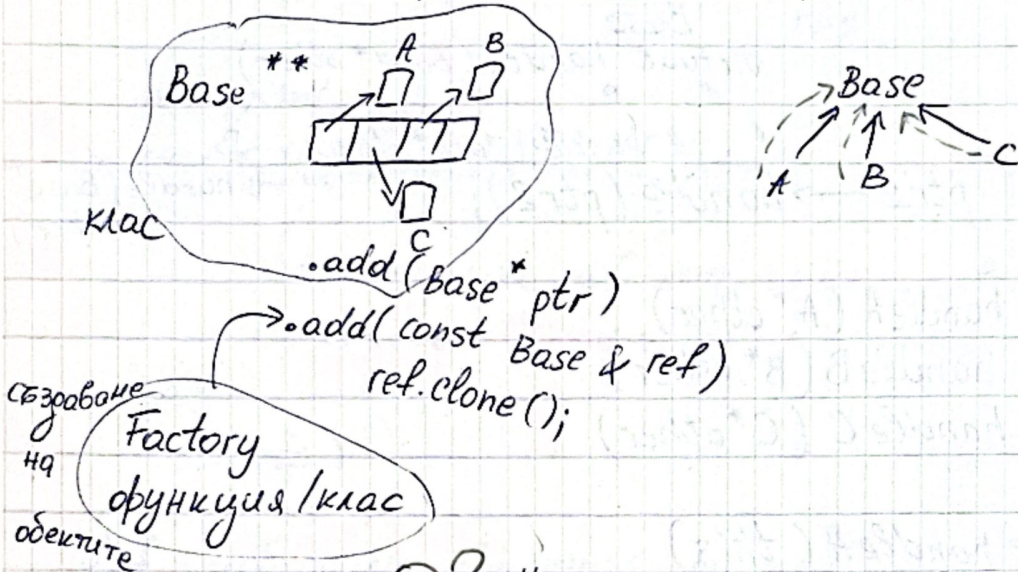


20.05.2024г.

Лекция 13

f(const A&) - заката lvalue и rvalue
 // Ако беше без const биқа му
 f(rvalue)

Хетерогенен контейнер

① collection c;
 c.add(factory(param)); // този случай е по-добър при него collection

collection c;
 ② c.add(param) > collection::add(param)
 factory(param)

$f(A * ptr1, C * ptr2)$

→ конкретна логика, която зависи от типа и на двата обекта.

$A \leftrightarrow B$
 $B \leftrightarrow A$
 $C \leftrightarrow A$

} // различни логики

Visitor
Pattern

$Base$
 $virtual\ handle(Base * other)$
 $\uparrow$
 $A::handle(Base * other)$
 $\uparrow$ знаем че ptr1 е обект от тип A
 $ptr1 \rightarrow handle(ptr2);$
 $\downarrow$
 $B::handle(Base * other)$

- $virtual\ handleA(A * other)$
- $virtual\ handleB(B * other)$
- $virtual\ handleC(C * other)$

$other \rightarrow handleA(this)$

$C::handleA(A * other)$
 $< C, A >$

⚠ Проблеми:

- 1) Родителя знае за наследниците си.
forward declaration // решението

Base.h

```

class A;
class B;
class Base;

```

→ обещавам, че при linking-а ще има такъв клас

използваме
Можем да правим само указатели и референции

- 2) Всеки нов наследник
→ нова виртуална функция

Шаблони - Тема 12 и 13

клас / функция с общо предназначение спрямо типа

```
template < typename T >
```

class

```
const T& max(const T& lhs, const T& rhs)
```

```
return lhs < rhs ? lhs : rhs;
```

```
max <int> (3, 9)
```

```
max <string> ("ABC", "XY");
```

→ генерира нов код, където T е заместено,
със съответния тип

~~max(3, 9)~~

def: Необходима функция на тип.

```
class A { ... }
```

```
A obj1, obj2;
```

```
max < A > (obj1, obj2) // компилационна грешка
```

→ такава A не има
дефиниран оператор <

! Всички колекции трябва да имат дефиниран оператор <

шаблон = параметричен полиморфизъм

compile-time

// защото функциите/класовете се генерират
по време на компилация

13A
• queue
(динамична)

13B
stack
↑
горна граница на елементите ще има

Пример stack.

→ Абстрактна структура от данни

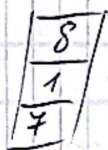
→ имаме дефинирано поведение // знаем какво трябва да стане, но нямаме дефинирана имплементация

stack <int> s

s.push(7)

s.push(1)

s.push(8)



s.pop() // 8

s.pop() // 1

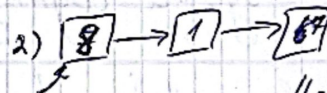
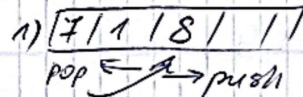
s.pop() // 7

• максимален капацитет

→ Капацитетът да е шаблон

→ Типа на стека също е шаблон

Възможни имплементации



// свързан списък

```
template <class T, unsigned N>
class Stack {
    T T data[N];
};
```

stack <int, 33> S1;

stack <int, 7> S2;

stack <char, 3> S3;

} правим три различни стека

⚠ Проблем със разделната компилация

```
template <class T>
class A { ... }
```

→ A.h

A.cpp

main.cpp

```
A <int> c1;  
A <char> c2;
```

компилира се

компилира се

A.cpp -----> A.h

main.cpp -----> A.h

A <int>
:

class A <char>

Това става по време на компилация и A.cpp не го вижда

// На A.cpp не вижда това, няма да генерира.
Той има шаблонния код A.h само.

Решение на проблема:

1) Не използваме разделна компилация // всичко е в един клас

A.h

```
template <typename T>  
class A { void f(); ... };
```

```
template <typename T>  
void A <T>::f() { ... }
```

2) Инстанциране на типове

A.h

A.cpp

// Имплицитно ще кажем с какви типове
ще използваме класа

```
template <typename T>  
class A { ... };
```

```
template <typename T>  
A <T>::f() ...
```

```
template A <int>
```

```
template A <char>
```

с какви типове
ще използваме класа;

с какви типове да генерира код

13096
T arr[N];
задавам T да има деф. конструктор

Темплейтна специализация

→ Клас / функция с различно поведение спрямо типа
(обикновено е за конкретен тип)

```
template <class T>
```

```
sort(T* arr, size_t size)
```

Quick sort + Insertion sort

→ темплейтна специализация за char*

```
template < >
```

```
sort(char* arr, size_t size)  
countSize
```

```
sort<int> (...)
```

```
sort<char> (...)
```

⚠ можете да добавяте член-функции на клас
само за конкретен тип

Пример:

```
Vector <T> [T|T|T|T|...]
```

Темплейтна специализация за bool

```
template <
```

```
Vector <bool> {
```

Битсет

```
}
```

~~[bool|bool|bool|bool]~~

// за домашното:

Обекти, които се върнат като функции

задача! Клас Average - 2 целочислени функции

- (9) → средноаритметичното на резултатите от двете функции

Average f(id, [] (int x) { return 0; })

f(10) → 5

f(11) → 5.5

Демонстрирайте използването на вашия клас със следните 2 функции:

1) $f(x) = x$

2) Потребителят въвежда три числа a, b, c

правим клас и трябва да се върне като функция

$$f(x) = \begin{cases} x, & x=a \text{ или } x=b \text{ или } x=c \\ 0, & \text{в противен случай} \end{cases}$$

```
class MyFunc {
```

```
    int a;
```

```
    int b;
```

```
    int c;
```

} контекст на функцията

```
    operator()(int x) {
```

```
        if (x==a || x==b || x==c)
```

```
            return x;
```

```
        return 0;
```

```
    }
```

MyFunc f(3, 9, 6) → обект, който се върне като функция

Average fa(id, f)

// Класът Average ще е шаблонен

Масив от различни функции:

Function

virtual operator()=0;

↑
template < typename T >

Func With Content {

T context;

operator() override

{ &... }

}

// шаблонна функция
наследници

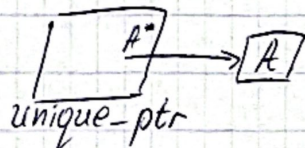
Умни указатели

→ обвиващи класове на указателите,
които менижират паметта на обектите, към които
сочат.

`new` ↔ `delete` // за го ограничим използването

`auto_ptr <A> ptr(new A());`
`deprecated;` , заради новите 3 указателя

1) `Unique_ptr`
точно 1 указател за точно 1 обект



В деструктора изтривате обекта

⚠ Забранени са копиращ конструктор
и оператор `=`

`make_unique` — обвиваща функция на `new`
създаване `new` имплицитно
{ `unique_ptr <A> = make_unique <A>(3,4,5);`
`new A(3,4,5);`

{ създаване `delete;`